

15 pièges d'agrégation SQL

qui faussent tes totaux

Le guide pour ne plus présenter un faux chiffre en réunion

La requête tourne, aucune erreur, le chiffre s'affiche. Et il est faux. C'est le pire moment : tu présentes un total en réunion, et quelqu'un dit « ce chiffre est faux ». L'agrégation ne plante presque jamais : elle additionne des doublons, ignore des `NULL`, mélange des grains, en silence. Voici les 15 pièges qui font dériver un total, chacun avec le symptôme, la conséquence et la correction.

■ ■ ■ GROUP BY : le regroupement qui ment

1. GROUP BY partiel : une colonne du SELECT n'est ni agrégée ni regroupée.

Le piège : Tu ajoutes `region` au `SELECT` pour « avoir le détail », mais tu l'oublies dans le `GROUP BY`.

La conséquence : Sur PostgreSQL, SQL Server et MySQL en mode strict, la requête est rejetée. Mais si elle passe (MySQL `ONLY_FULL_GROUP_BY` désactivé), le moteur affiche une valeur arbitraire de `region` pour chaque groupe : ton total par client devient un total par client agrémenté d'une région prise au hasard, donc ininterprétable.

La correction : Toute colonne non agrégée du `SELECT` doit figurer dans le `GROUP BY`. Si tu ne veux pas du détail, retire-le du `SELECT` ; si tu le veux, regroupe dessus.

2. HAVING ou WHERE : filtrer sur un agrégat.

Le piège : Écrire `WHERE SUM(montant) > 1000` pour ne garder que les gros clients.

La conséquence : Le `WHERE` est évalué avant le regroupement : l'agrégat n'existe pas encore, le moteur refuse. À l'inverse, mettre dans le `HAVING` un filtre ligne à ligne (`HAVING pays = 'FR'`) fonctionne mais s'applique après l'agrégat : tu agrèges la France ET l'étranger, puis tu jettes des groupes, au lieu d'agréger la France seule.

La correction : `WHERE` filtre les lignes avant l'agrégat, `HAVING` filtre les groupes après. Un critère sur une colonne brute va dans `WHERE`, un critère sur `SUM/COUNT` va dans `HAVING`.

3. COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col).

Le piège : Compter « le nombre de clients » avec `COUNT(*)`.

La conséquence : Les trois ne comptent pas la même chose, et aucun ne lève d'erreur :

- `COUNT(*)` : toutes les lignes, `NULL` compris.
- `COUNT(col)` : les lignes où `col` n'est pas `NULL`.
- `COUNT(DISTINCT col)` : les valeurs uniques non-`NULL`.

La correction : Un client présent sur 4 commandes compte 4 fois dans COUNT(*) et 1 fois dans COUNT(DISTINCT client_id). « Nombre de clients » = COUNT(DISTINCT).

4. SUM gonflé par une jointure qui duplique les lignes.

Le piège : Joindre commandes à lignes_commande puis faire SUM(commandes.montant).

La conséquence : Une commande à 3 lignes est dupliquée 3 fois par la jointure : son montant est additionné 3 fois. Ton chiffre d'affaires est gonflé, sans erreur ni avertissement. C'est la cause n°1 des totaux trop hauts.

La correction : Pré-agrège chaque table à son grain dans une CTE avant de joindre. Évite SUM(DISTINCT montant) : il supprime aussi les montants légitimement égaux et fausse le total dans l'autre sens.

```
WITH par_cmd AS (  
  SELECT commande_id, SUM(qte*pu) AS total  
  FROM lignes_commande  
  GROUP BY commande_id  
)  
SELECT SUM(total) FROM par_cmd;
```

■ ■ ■ Moyennes : le chiffre qui ment le plus

5. AVG d'agrégats déjà calculés : la moyenne de moyennes.

Le piège : Calculer un panier moyen par magasin, puis faire la moyenne de ces moyennes pour avoir le panier moyen global.

La conséquence : Une moyenne de moyennes n'est juste que si chaque groupe a le même poids. Un magasin avec 5 ventes pèse autant qu'un magasin avec 5 000 ventes : le résultat est faux, souvent de plusieurs pour cent.

La correction : Repars des lignes brutes : SUM(montant) / SUM(quantite), ou pondère explicitement par l'effectif de chaque groupe. Ne fais jamais AVG() d'une colonne qui est déjà une moyenne.

6. AVG : les 0 comptent, les NULL sont ignorés.

Le piège : Croire que AVG traite pareil une valeur absente (NULL) et une valeur nulle (0).

La conséquence : AVG ignore les NULL au numérateur ET au dénominateur : ils ne comptent pas. Un 0, lui, est une vraie valeur : il est additionné et compté, donc il tire la moyenne vers le bas. La même colonne donne deux moyennes très différentes selon que les ventes vides sont stockées en 0 ou en NULL.

La correction : Décide du sens métier. Pour inclure les jours sans vente : COALESCE(montant, 0) avant l'AVG. Pour ne moyenner que les jours actifs : laisse les NULL.

7. MIN / MAX et les NULL : le piège est l'inverse de ta crainte.

Le piège : Craindre qu'un seul NULL dans la colonne rende MIN ou MAX égal à NULL.

La conséquence : Faux : comme toutes les fonctions d'agrégation, MIN et MAX ignorent les NULL. Ils ne renvoient NULL que si le groupe est entièrement NULL ou vide. Le vrai piège est inverse : tu crois que MIN(date) couvre toutes les lignes, alors qu'il saute silencieusement celles à date manquante.

La correction : Si une date NULL doit compter (commande non datée à traiter), filtre-la ou traite-la à part : l'agrégat ne te la signalera pas.

8. GROUP BY sur une expression : regrouper sur autre chose que ce que tu crois.

Le piège : Afficher le mois avec TO_CHAR(date, 'Mon') et regrouper dessus.

La conséquence : Tu regroupes sur le libellé du mois, sans l'année : janvier 2024 et janvier 2025 tombent dans le même groupe. Sur plusieurs années, tes totaux mensuels additionnent des périodes différentes.

La correction : Regroupe sur une expression qui isole vraiment la période, par exemple DATE_TRUNC('month', date), qui conserve l'année. Vérifie toujours que l'expression du GROUP BY est aussi fine que le grain visé.

■ ■ ■ Grain et doublons : ce que le total recompte

9. Filtrer avant ou après l'agrégat : tu ne changes pas le même chiffre.

Le piège : Penser que filtrer sur les statuts « avant » ou « après » l'agrégat donne le même total.

La conséquence : Un WHERE statut = 'payé' retire les lignes non payées de la population avant l'agrégat : COUNT(*) et SUM ne portent que sur le payé. Un HAVING qui filtre après garde tout dans l'agrégat puis jette des groupes : le total par groupe inclut encore les lignes que tu croyais exclues.

La correction : Décide ce qui doit entrer dans le calcul. Pour exclure des lignes du total : WHERE. Pour ne garder que certains résultats agrégés : HAVING. Les deux peuvent coexister, ils ne font pas le même travail.

10. Division entière dans une moyenne maison.

Le piège : Recalculer une moyenne à la main avec SUM(x) / COUNT(*) sur des entiers.

La conséquence : Entier divisé par entier : le résultat est tronqué sous PostgreSQL, SQL Server, SQLite et DB2 (7/2 donne 3, pas 3,5), alors que MySQL renvoie 3,5 avec / (seul DIV y tronque). Pire, AVG(colonne_entiere) tronque sous SQL Server mais renvoie un décimal sous PostgreSQL : le même code donne deux résultats selon le moteur.

La correction : Caste avant la division : SUM(x) * 1.0 / COUNT(*), ou CAST(x AS

numeric) dans l'AVG. Ne te fie jamais au type par défaut pour un calcul de moyenne.

11. DISTINCT qui masque des doublons : la liste paraît propre, le total reste faux.

Le piège : Voir des lignes en double, ajouter `SELECT DISTINCT`, et croire le problème réglé.

La conséquence : `DISTINCT` dédoubleonne l'affichage des lignes, mais ne touche pas la cause (souvent une jointure qui duplique, voir piège 4). La liste paraît nette, et pourtant un `SUM` calculé sur la même requête additionne toujours les doublons sous-jacents : tu corriges ce que tu vois, pas ce qui est compté.

La correction : Remonte à la source de la duplication (clé de jointure non unique, grain trop fin) et corrige-la. `DISTINCT` est un pansement d'affichage, pas une correction d'agrégat.

12. Mélange de grains dans un même SELECT.

Le piège : Afficher dans la même ligne un total client (`SUM` par client) et un total global, ou un détail de ligne à côté d'un agrégat.

La conséquence : Deux mesures à des grains différents (par ligne, par client, global) ne se lisent pas sur la même ligne sans définir lequel domine. Le moteur applique le `GROUP BY`, et la colonne au mauvais grain affiche une valeur qui ne correspond ni au détail ni au total : le lecteur additionne mentalement des chiffres incompatibles.

La correction : Fixe un seul grain par requête. Pour comparer un détail à un total, calcule le total dans une window function (`SUM(x) OVER ()`) ou une CTE séparée, sans écraser le grain de base.

■■■ Compteurs et sous-totaux : les pièges experts

13. COUNT vs SUM d'un booléen : compter les « vrais ».

Le piège : Compter les commandes livrées avec `COUNT(est_livree)`.

La conséquence : `COUNT(colonne)` compte toutes les lignes où la colonne n'est pas `NULL` : les `TRUE` ET les `FALSE`. Tu obtiens le nombre de commandes renseignées, pas le nombre de livrées. Le chiffre paraît plausible, donc personne ne le remet en cause.

La correction : Pour compter les vrais, additionne-les :

```
-- portable
SUM(CASE WHEN est_livree THEN 1 ELSE 0 END)
-- PostgreSQL
COUNT(*) FILTER (WHERE est_livree)
```

14. ROLLUP mal lu : recompter les sous-totaux et confondre les NULL.

Le piège : Faire un `GROUP BY ROLLUP(region, ville)` puis resommer la colonne du résultat.

La conséquence : `ROLLUP` ajoute des lignes de sous-total (par région) et une ligne de total général, en plus des lignes détaillées. Resommer la colonne compte donc deux fois les montants. Et ces lignes de synthèse portent un `NULL` dans les colonnes de regroupement roulées (region, ville) : impossible de les distinguer d'un `NULL` de donnée à l'œil.

La correction : Ne resomme jamais un résultat de `ROLLUP`. Pour repérer les lignes de synthèse, utilise `GROUPING(ville) = 1` (1 = colonne roulée, donc sous-total) plutôt que de tester le `NULL`.

15. Agrégat sans clé de regroupement : tout s'effondre sur une ligne.

Le piège : Écrire `SELECT client_id, SUM(montant) FROM ventes` sans `GROUP BY`.

La conséquence : Sans `GROUP BY`, l'agrégat porte sur toute la table : une seule ligne de résultat. PostgreSQL et SQL Server rejettent `client_id` dans ce `SELECT`. Mais MySQL avec `ONLY_FULL_GROUP_BY` désactivé accepte : il colle un `client_id` pris au hasard à côté du grand total, et tu repars avec un chiffre qui ressemble à un total par client.

La correction : Dès qu'une colonne brute côtoie un agrégat, ajoute le `GROUP BY` correspondant. Garde `ONLY_FULL_GROUP_BY` actif sous MySQL : c'est le garde-fou qui transforme ce total muet en erreur visible.

■ ■ ■ Le réflexe à retenir

Un total qui s'affiche n'est pas un total juste. Avant de le présenter, vérifie trois choses :

- **le grain** : tes jointures dupliquent-elles des lignes ?
- **les NULL** : sont-ils comptés comme tu le crois ?
- **le regroupement** : chaque colonne brute a-t-elle son `GROUP BY` ?

Garde ce guide sous la main : ces 15 pièges sont ceux qui te font présenter un faux chiffre en réunion.